Code cleanup and untangling: unused-code evidence and change boundaries

Strengths and best use

Dead-code detection and architectural untangling complement one another. Static tools identify suspicious unused files, exports, dependencies, or Python symbols. Architectural review asks whether the code's boundaries match the way behavior changes. A cleaner dependency graph can make both human and agent work easier to review. Knip (https://knip.dev/overview/getting-started) - Vulture (https://github.com/jendrikseipp/vulture)

Establish evidence before removal

Create a branch and record the behavior and checks that matter. Inventory entry points, generated files, dynamic imports, framework conventions, plugins, and runtime configuration. Static reports are candidates for investigation. A dynamically referenced symbol or deployment-only file may appear unused. Make a small removal, inspect the diff, and exercise the relevant behavior before continuing.

The current Knip v6 documentation requires Node.js 20.19 or newer, or Bun. Use its getting-started instructions for the selected package manager. For a Python exploration environment, Vulture documents:

```
python -m pip install vulture
vulture . --min-confidence 100
```

Even high-confidence output needs context and verification. Preserve a rollback route and keep dependency cleanup separate from unrelated behavioral changes.

Use the published Untangle skill carefully

An actual source is available: Caldo publishes an MIT-licensed Untangle Codebase skill. This corrects the dossier's unresolved installation link. The checked repository documents:

```
npx skills add caldoai/skills --skill untangle-codebase
```

Repository (https://github.com/caldoai/skills) - Skill contents (https://github.com/caldoai/skills/blob/main/skills/untangle-codebase/SKILL.md)

Its central question is whether pieces of code should change together. Look for hidden coupling, unnecessary links, and surplus machinery. Review the skill before installation: its scope includes broad refactoring and optional cleanup of comments or tests. Specify ownership, allowed files, preserved behavior, and review checkpoints. The association with a creator's Maze project was not independently established by these sources.

Evaluate one boundary

Choose one recurring change that crosses several modules. Trace inputs, outputs, shared state, error handling, and tests. Move one responsibility or simplify one interface, then compare the files touched by the next representative change. Count changed modules, regressions, review time, and setup effort. Repeated agent recommendations are useful leads but need the same concrete evidence as any other refactor proposal.

Reusable prompt

```
Inspect this bounded area for unused code and change coupling. Identify runtime entry points and evidence before recommending removal. Explain which pieces should change together, propose one reversible boundary improvement, preserve behavior, and compare relevant checks and review effort before and after.
```

Project idea

Create a cleanup evidence ledger linking each candidate to its caller search, runtime relevance, decision, diff, verification, and rollback. Pilot one module before broad automation.

Original sources

- https://www.instagram.com/reel/Deb0R3wt9el/
- https://www.instagram.com/reel/DfYbglQxEbl/